

Draw It or Lose It

Software Design Document

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 4
Domain Model 4
Evaluation 6
Recommendations 9

Document Revision History

Version	Date	Author	Comments
1.0	09/20/26	Angela Grumbach	Worked on the executive summary, requirements, and design constraint sections.
1.2	10/4/26	Angela Grumbach	Completed the Evaluation table, examining Server Side, Client Side and Development Tools for Mac, Windows, Linux and Mobile Devices.
1.3	10/18/26	Angela Grumbach	Updated Server Side MacOS section. Completed all six sections under recommendations.

Executive Summary

The Gaming Room currently offers Draw It or Lose It as an Android application, which limits the game to users of that platform. The client would like to expand the game into a web-based application that can be accessed across multiple operating systems and device types. Moving to a web-based environment will allow players using different devices to access the same game and participate together as members of a team.

Two of the client's requirements are that a game will be able to have multiple teams and that each team can have multiple players. The Game class contains a list of teams, allowing multiple teams to be added to a game without changing the structure of the class. Similarly, the Team class contains a list of players, allowing multiple players to be assigned to each team. Using lists will make the application flexible because teams and players can be added as needed.

The client wants game and team names to be unique and have the ability to check if a name is being used. The getGame(name) method allows the application to search the existing games to determine whether a name is already in use. If the name exists, the application can prevent a duplicate from being created and require the user to choose a different name.

To manage access to the game service and ensure only one instance of GameService exists within the running application, the application can use the Singleton design patter. The Singleton pattern prevents other classes from creating additional instances of GameService. The Singleton pattern controls the creation of GameService by preventing other classes from creating additional instances.

After evaluating the available operating platforms, I recommend Linux as the server operating system for Draw It or Lose It. Linux supports the Java application and provides the flexibility needed for a web-based game that can be accessed from different devices. I also recommend using Amazon S3 for image storage and Amazon RDS with MySQL for player and game information. These services would allow The Gaming Room to manage its resources as the application expands.

Requirements

The client has four software requirements:

- A game will have the ability to have one or more teams involved.
- Each team will have multiple players assigned to it.
- Game and team names must be unique to allow users to check whether a name is in use when choosing a team name.
- Only one instance of the game can exist in memory at any given time.

Design Constraints

1. **Cross-platform compatibility:**

   Because Draw It or Lose It will be developed as a web-based application, the software can't rely on features that are specific to Android or another operating system. Platform-specific features could cause the application to function differently or not work correctly for some users. Developing the game to operate through a web browser will provide a more consistent experience and allow players using different devices and operating systems to participate the same game.

2. **Multi-user environment:**

   Because multiple players on different devices can participate in the same game, the application must maintain a consistent game state for all users. If each device maintains separate game information,

players could receive conflicting information, like multiple players believing they had won the same round. The web-based application should manage game information so that changes to the game are reflected consistently for all participants.

3. **Unique data and instance management:**

Each game, team and player must have a unique identifier so that the application can distinguish between individual objects. For example, two players may have the same name, but their unique IDs allow the application to identify each player separately and maintain the correct data for each one. Game and team names must also be checked before creation to prevent duplicate names from being used. The Singleton design pattern will ensure that only one instance of the game can exist in memory at any given time, preventing players from creating separate instances and helping ensure that they are accessing the same game information.

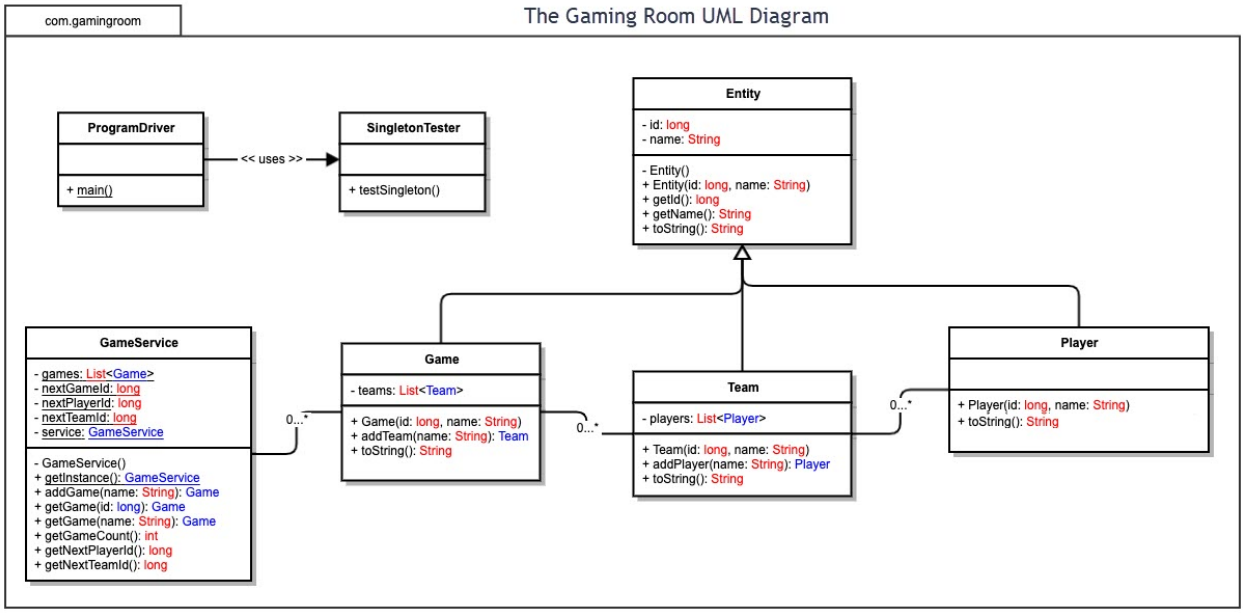
System Architecture View

The Gaming Room's Draw It or Lose It application will use a client-server architecture that allows players on different operating systems and devices to access the same game through a web browser. The Java application will be hosted on a Linux server, while Amazon S3 will provide storage for the game's image library and Amazon RDS will manage player and game information. These components will communicate over a network to support multiple players and games.

A detailed system architecture diagram is outside the scope of this proposal. However, the recommended architecture provides a foundation for expanding the application to support additional users and devices.

Domain Model

The UML diagram shows how Entity is a parent class of Game, Team, and Player. It contains common id and name attributes that Game, Team, and Player inherit. This demonstrates the OOP principle of inheritance, which reduces duplicate code by allowing common attributes to be defined in one location. The Game class contains a list of teams (List<Team>) which allows a game to have several teams. Similarly, each team contains its own list of players (List<Player>) which allows a team to have several players. The application can add teams and players as needed without changing the basic structure of the classes.
There are several attributes which are private, for example (- teams: List<Team>), the - indicating that they are private. The data within these objects is private but the classes provide public methods for interacting with the data. This demonstrates the OOP principle of encapsulation. Doing this gives the application more control over changes to its data and can help prevent invalid or duplicate information from being added.
GameService manages games and unique IDs. We use a Singleton pattern on GameService to allow different parts of the application to access GameService but they all receive the same instance. The constructor is private, preventing the other classes from creating additional instances of GameService. Other classes can call getInstance() to retrieve the single instance. This allows the application to maintain one central game service rather than creating separate services with different game information. GameService uses nextGameId, nextTeamId and nextPlayerId to assign unique identifiers as new objects are created. Incrementing these values allows each game, team, and player to be identified separately.


com.gamingroom
The Gaming Room UML Diagram
ProgramDriver
+ main()
<< uses >>
SingletonTester
+ testSingleton()
Entity
- id: long
- name: String
- Entity()
+ Entity(id: long, name: String)
+ getId(): long
+ getName(): String
+ toString(): String
GameService
- games: List<Game>
- nextGameId: long
- nextPlayerId: long
- nextTeamId: long
- service: GameService
- GameService()
+ getInstance(): GameService
+ addGame(name: String): Game
+ getGame(id: long): Game
+ getGame(name: String): Game
+ getGameCount(): int
+ getNextPlayerId(): long
+ getNextTeamId(): long
0...*
Game
- teams: List<Team>
+ Game(id: long, name: String)
+ addTeam(name: String): Team
+ toString(): String
0...*
Team
- players: List<Player>
+ Team(id: long, name: String)
+ addPlayer(name: String): Player
+ toString(): String
0...*
Player
+ Player(id: long, name: String)
+ toString(): String

Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac computers are capable of running Java and web server software, but macOS would not be a recommended server operating system for The Gaming Room. In 2022, Apple discontinued macOS Server, which means the company no longer offers its dedicated server application for managing server services. Although macOS can still run some server software, the discontinuation of macOS Server makes it a less practical choice for a company planning to host and expand a large-scale web application.	Linux is commonly used for hosting large-scale web applications and would be a good fit for Draw It or Lose It. Linux can often be used with our an operating system licensing fee, which can reduce costs as additional servers are added. Linux also supports a wide range of hardware and server software. A disadvantage is that Linux may require employees with Linux-specific experience to configure, maintain and secure the servers.	Windows server can host a web-based application and it's commonly used in business environments. It supports Java and web server software and can run on many different types of server hardware. Windows provides familiar graphical administration tools which may make managing the server easier for developers or administrator. The disadvantage is that Windows requires paid licensing which can increase costs as the application grows and additional servers are needed. For large-scale applications these licensing costs could make Windows more expensive.	Mobile operating systems such as Android or iOS are not designed to host large-scale web applications. Mobile devices have limited processing power, memory storage, and network bandwidth compared with dedicated servers. They also rely on battery power and network connections that may change or become unavailable. A mobile device could technically run some server functions, it would not be practical or reliable for supporting thousands of players. Mobile devices should be used as clients that connect to the application rather than the server hosting it.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Client Side	Mac users can access Draw It or Lose It through a supported web browser without requiring a separate macOS application. Using a responsive HTML interface reduces development time and cost because much of the same code can be used across operating systems. Multiple browsers would need to be tested, including Safari, to make sure the interface and game functions work correctly. Browser compatibility and different screen sizes should be considered during development.	Linux users can access the game through a supported web browser without requiring a separate Linux application. Because the application uses a browser-based interface, the same web application can be accessed from Linux and other desktop operating systems. Again developers should test the application on major browsers rather than develop a separate version for each Linux distribution.	Windows users can access the game through supported web browsers without requiring a separate Windows application. The HTML interface allows the same code to be used across Windows, Mac and Linux, reducing development time and cost. The application would need to be tested on common windows browsers such as Edge, Chrome and Firefox to make sure the interface and game functions work consistently. This approach requires less platform-specific expertise and developing a separate Windows application.	The Gaming Room currently supports Android devices, but the web-based application will expand support to both Android and iOS devices. The new interface must use responsive design so that it adjusts to different screen sizes and orientations and works correctly with touch controls. Developers will need to test the application on common mobile browsers, including Safari on iOS and Chrome on Android. Supporting different devices and screen sizes will require additional testing time and expertise, but using a browser-based interface will allow the company to expand beyond its existing Android application without having to develop and maintain a separate native application for every mobile platform.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Development Tools	Developers using macOS can use Java with a development tools such as Eclipse and Maven to build and manage the server-side application, along with HTML, CSS and Java for the web interface. Many of these development tools are available without additional licensing costs. macOS can also run Xcode, which provides tools for testing compatibility with Apple devices. Because most of the development tools used for the web application are cross-platform, a separate macOS development team wouldn't be necessary although developers with experience testing Safari and Apple devices would be helpful.	Developers using Linux can use Java with tools such as Eclipse and Maven, along with HTML, CSS, and JavaScript for the web interface. Many Linux distributions and development tools are free and open source, which can keep licensing costs low. Because Java and the web development tools are cross-platform, a separate Linux development team would not be necessary. However, developers who are unfamiliar with Linux may require additional training or experience to work efficiently in the environment.	Developers using Windows can use the same tools as with Mac and Linux. Windows also supports development tools such as Visual Studio. Many of these tools are available without additional licensing costs, although Windows itself and some professional development software may have licensing costs. Because Java and web development tools are cross-platform, a separate Windows development team would not be necessary. Developers would still need to test the application on common Windows browsers, particularly Edge, Chrome, and Firefox.	The browser-based application can be developed using standard web technologies such as HTML, CSS and Java, reducing the need for separate Android and iOS development teams. Android Studio can be used to test Android devices, while an iOS simulator can be used for iOS testing. Many of the development tools are available at no additional licensing cost, but the supporting Android and iOS will require additional testing time and knowledge of the difference between mobile devices and browsers.

Recommendations

1. **Operating Platform**:

I recommend Linux as the server operating system for The Gaming Room's Draw It or Lose It application. Linux is commonly used to host web-based applications and supports Java, which is the programming language used for the game. Using Linux as the server operating system allows The Gaming Room to host the game in one central location while players access it through web browsers on different operating systems, including Windows, macOS, Linux, Android, and iOS.

One advantage of Linux is that many distributions are free and open source, which can help reduce licensing costs as the application grows. Linux also supports a wide range of hardware and server software giving The Gaming Room flexibility if additional servers are needed to support more players. One disadvantage is that Linux may require employees with Linux-specific experience to configure, maintain, and secure the servers. However, the lower licensing costs, flexibility, and ability to support a web-based application make Linux a good choice for The Gaming Room.

2. **Operating Systems Architectures**:

Linux uses an architecture that separates the kernel from applications running in user space. The kernel is the central part of the operating system and is responsible for managing system resources such as memory, processing, hardware, and network connections. Applications run separately from the kernel and request access to these resources when needed. This separation helps protect the operating system by preventing applications from directly accessing or changing critical system resources.

For The Gaming Room, the Java application and web server software would run in user space while the Linux kernel manages the resources needed to support the game. Linux can manage multiple processes at the same time, allowing the server to handle the game application and other system tasks. Linux also uses user accounts and file permissions to control access to system resources and protect important files from unauthorized changes. The operating system can be maintained through software updates and security patches to help keep the server secure and reliable. These features make Linux a good choice for hosting Draw It or Lose It as the application expands to support players using different devices.

3. **Storage Management**:

The Gaming Room will need a storage management system that can store the game's images and information while allowing multiple games to run at the same time. Draw It or Lose It has 200 high-definition images at approximately 8 MB each, requiring approximately 1.6 GB of storage for the image library. I would recommend using cloud storage, such as Amazon S3, to store these images separately from the Linux server running the game application. Amazon S3 provides persistent storage and can expand as the application's storage needs increase. Because the images do not change frequently, they can remain in cloud storage and be retrieved when needed during gameplay. This would also allow multiple games to access the same image library without requiring separate copies for each game.

The application will also need to store information that changes more frequently, such as player accounts, teams, game information, and scores. I would recommend using Amazon RDS with MySQL to manage this information. MySQL is a relational database that organizes information into tables that can be connected using unique identifiers. This would work well with the game's existing structure because games, teams, and players already have unique IDs that allow the application to distinguish between them. Amazon RDS would allow The Gaming Room to use a MySQL database in the cloud without having to manage a separate physical database server. AWS also provides features such as automated backups and database maintenance, which can help reduce the work required to manage the database. Using Amazon S3 for the image library and Amazon RDS for frequently changing information would allow The Gaming Room to manage both types of storage as the application grows.

4. **Memory Management**:

Linux uses memory management techniques to allocate memory to running applications and manage the available system resources. For Draw It or Lose It, the Linux server will need enough RAM to support multiple games running at the same time. The Java application also uses garbage collection to release memory occupied by objects that aren't needed anymore.

I recommend using caching to improve the performance of the game. Draw It or Lose It has approximately 1.6 GB of images stored in Amazon S3, but the entire image library does not need to be loaded into the server's memory. Instead, the application can temporarily cache images needed for active games and preload images for upcoming rounds. This reduces the need to retrieve the same images repeatedly from cloud storage and could help reduce delays between rounds. Images that are no longer needed can be removed from the cache to make memory available for other games.

The Gaming Room will also need to consider memory usage on the players' devices. Because the game will be accessed through a web browser, the client devices will only need enough memory to run the browser and display the images and game information being used. Most of the game's processing and storage management will take place on the server. As the number of players increases, The Gaming Room should monitor the Linux server's memory usage and increase its available resources when necessary to maintain consistent performance.

5. **Distributed Systems and Networks**:

Draw It or Lose It will need a network structure that allows players using different devices to connect to the same game. I would recommend using a client-server architecture where the Java application runs on a central Linux server and players access the game through their web browsers. The application can use REST APIs and HTTPS to communicate between the clients and the server. When a player submits a guess or requests game information, the browser sends a request to the server. The server processes the request and sends a response back to the browser. This would allow players using Windows, Mac, Linux, and mobile devices to participate in the same game without needing separate versions of the application.

The Linux server will also communicate with Amazon S3 to retrieve game images and Amazon RDS to access and update game information. Using cloud storage and a cloud database would allow these resources to be managed separately from the game application. The application will need to keep

game information updated so that players can see the current round, scores, and remaining time. It should also be able to handle temporary network interruptions by allowing players to reconnect and retrieve the current game information. As the number of players increases, The Gaming Room can expand its server resources to support additional network traffic and maintain consistent performance.

6. **Security**:

Security will be an important consideration for The Gaming Room because Draw It or Lose It will be accessed by players using different devices over the internet. I would recommend using HTTPS to encrypt information transmitted between the players' browsers and the Linux server. This would help protect information such as login credentials and game activity while it travels across the network. The application should also use authentication to verify the identity of players and authorization to control what information they can access or change. For example, players should be able to participate in their own games but should not be able to modify another team's scores or access another player's account.

Linux provides several security features that can help protect the server and the application. User accounts and file permissions can restrict access to important system resources, while firewalls can limit unnecessary network connections. Regular software updates and security patches would also be needed to help protect the server from known vulnerabilities. Because The Gaming Room will be using Amazon S3 and Amazon RDS, access to the image library and database should also be restricted to authorized users and applications. Sensitive player information should be protected through encryption, and database backups should be maintained to help prevent permanent data loss. Using these security measures together would help The Gaming Room protect its application and player information as the number of users increases.